LAPORAN PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

PEKAN KE 6

Disusun Oleh :

Muhammad Irfan 2511531008

Kelas : IF B

Dosen Pengampu : Afdhal Dinilhak, M.Kom

Asisten Praktikum : Ezza Addini



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

TAHUN 2026

A.output tools.NimParams 2511531008

Parameter tugas untuk NIM 2511531008

CARD_ADMIN_PERCENT = 3

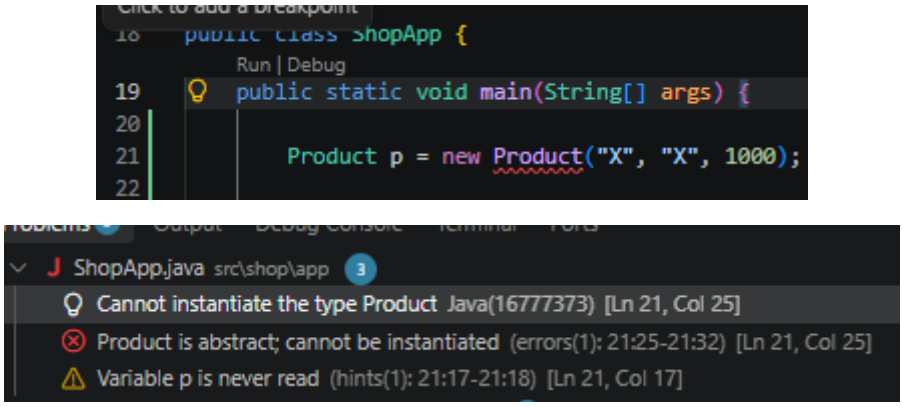
TRANSFER_ADMIN_FEE = 4000

SMS_MAX_LENGTH = 108

B. Tangkapan layar error compiler

L1. membuat objek Product

Menambahkan Product p = new Product("X", "X", 1000); ke fungsi main

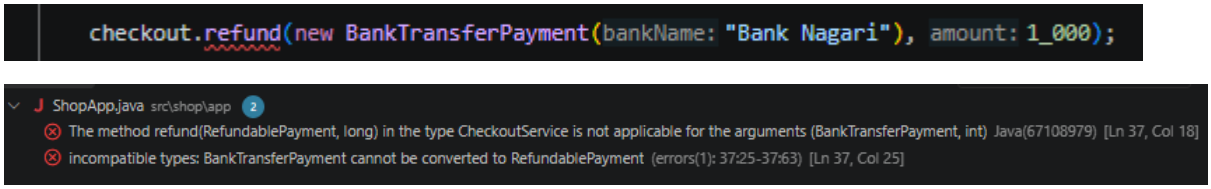


Gambar 1: Compile Error di L1

Alasan Error:

karena Product adalah abstract class yang punya abstract method category() tanpa isi, sehingga objeknya tidak boleh dibuat langsung. Objek hanya bisa dibuat dari subclass konkret seperti PhysicalProduct atau DigitalProduct.

L3 refund lewat transfer bank



Alasan Error:

CheckoutService.refund(...) hanya menerima RefundablePayment, sedangkan BankTransferPayment hanya mengimplementasikan PaymentMethod. Compiler menolak refund lewat transfer bank sebelum program berjalan, tanpa perlu if pengecekan.

L4. override calculateCost

```java
package shop.shipping;

public class FreeShipping extends ShippingService {
    public final long calculateCost(int weightGrams){
        if(weightGrams <= 0){
            return 0;
        }
        long kilograms = (weightGrams + 999L) / 1000;
        return kilograms * ratePerKg();
    }

}
```

```
Problems 6  Output  Debug Console  Terminal  Ports
FreeShipping.java src\shop\shipping 6
FreeShipping is not abstract and does not override abstract method ratePerKg() in ShippingService (errors(1): 3:14-3:26) [Ln 3, Col 14]
The type FreeShipping must implement the inherited abstract method ShippingService.ratePerKg() Java(67109264) [Ln 3, Col 14]
Implicit super constructor ShippingService() is undefined for default constructor. Must define an explicit constructor Java(134217868) [Ln 3, Col 14]
calculateCost(int) in FreeShipping cannot override calculateCost(int) in ShippingService (errors(2): 4:5-4:54) [Ln 4, Col 5]
    overridden method is final
Cannot override the final method from ShippingService Java(67109265) [Ln 4, Col 23]
```

Alasan :

Method calculateCost di ShippingService bertanda final, sehingga tidak boleh di-override oleh subclass mana pun. Ini mengunci rumus ongkir (pembulatan berat ke atas per kg) agar kurir hanya bisa mengubah tarif lewat ratePerKg().

C.Jawaban pertanyaan

T1.a adminFee sebagai abstract method

Mencoba menambahkan long adminFree (long amount)

```java
package shop.payment;
/** Contract for every payment method. */
public interface PaymentMethod {
    String getName();
 /** @return true if the payment succeeds */
    boolean pay(long amount);
    long adminFee(long amount);
}
```

```
ShopApp.java src\shop\app 2
<anonymous shop.app.ShopApp$1> is not abstract and does not override abstract method adminFee(long) in PaymentMethod (errors(1): 65:9-65:73) [Ln 65, Col 9]
The type new PaymentMethod(){} must implement the inherited abstract method PaymentMethod.adminFee(long) Java(67109264) [Ln 65, Col 37]
BankTransferPayment.java src\shop\payment 1
The type BankTransferPayment must implement the inherited abstract method PaymentMethod.adminFee(long) Java(67109264) [Ln 3, Col 14]
CreditCardPayment.java src\shop\payment 1
The type CreditCardPayment must implement the inherited abstract method PaymentMethod.adminFee(long) Java(67109264) [Ln 3, Col 14]
EWalletPayment.java src\shop\payment 1
The type EWalletPayment must implement the inherited abstract method PaymentMethod.adminFee(long) Java(67109264) [Ln 3, Col 14]
```

Jawaban:

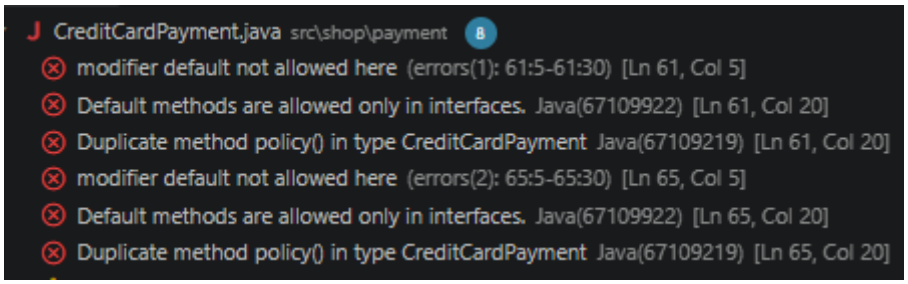
- Ada **4 error**, masing-masing 1 per file.

- Class yang rusak EWalletPayment, BankTransferPayment, CreditCardPayment, dan anonymous class voucher di ShopApp.
- Mengapa voucher di ShopApp juga ikut rusak? Anonymous class tetap sebuah class yang mengimplementasikan PaymentMethod, jadi semua method abstract-nya wajib diisi. Letaknya di ShopApp, bukan di package shop.payment, tapi aturannya sama.

T1.e Konflik diamond

Buatlah 2 default method dengan nama yang sama ke class CreditCardPayment

```
// in RefundablePayment
default String policy() {
    return "Refund maksimal 7 hari";
}
// in InstallmentPayment
default String policy() {
    return "Cicilan maksimal " + maxInstallmentMonths() + " bulan";
}
```



Alasan : CreditCardPayment mewarisi dua default method policy() dengan signature sama dari dua interface yang tidak saling berhubungan, sehingga compiler tidak tahu mana yang harus dipakai. Masalah ini diselesaikan dengan meng-override policy() di CreditCardPayment dan memanggil RefundablePayment.super.policy() serta InstallmentPayment.super.policy().

Jawaban Laporan

T1 Evolusi API dan konflik diamond

Jawaban untuk laporan :

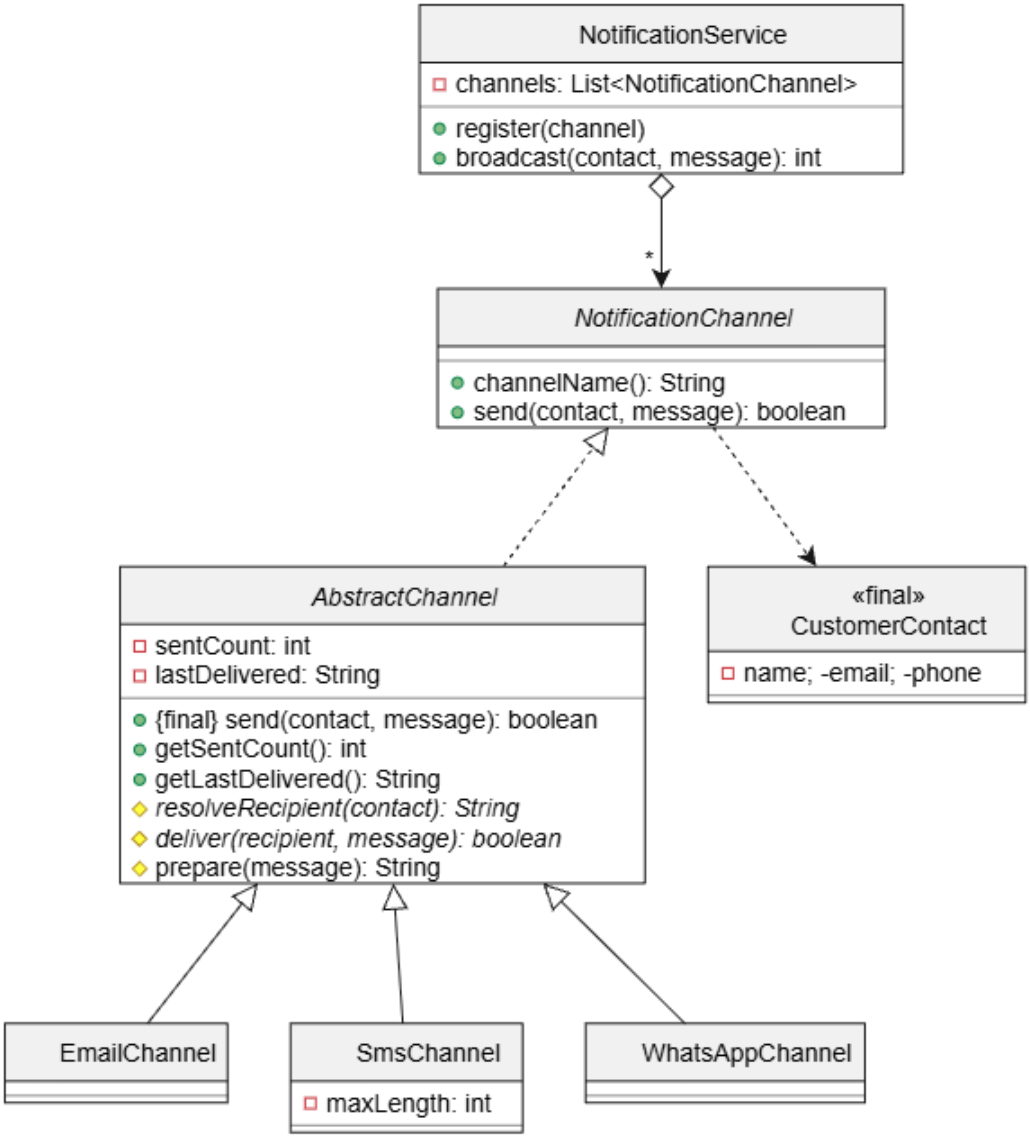
1) Mengapa EWalletPayment tidak perlu meng-override policy()?

Karena EWalletPayment hanya implements RefundablePayment, jadi hanya mewarisi **satu** default policy(). Tidak ada ambiguitas, sehingga Java langsung memakai versi itu (“Refund maksimal 7 hari”).

2) Mengapa Java tidak memilih salah satu default secara otomatis?

Karena kedua interface setara (tidak ada yang lebih spesifik dari yang lain), dan memilih diam-diam bisa menghasilkan perilaku yang salah dan tak terduga tanpa disadari programmer. Java lebih memilih **gagal saat kompilasi** dan memaksa programmer menentukan sendiri, apakah memakai salah satu, menggabungkan keduanya, atau menulis versi baru. Ini mencegah bug tersembunyi akibat urutan implements atau perubahan interface di kemudian hari.

T2 – Merancang sistem notifikasi pesanan



Keterangan :

Simbol	Warna	Visibilitas	Notasi UML	Di Java
Bulat	Hijau	Public	+	public
Kotak	Merah	Private	-	private
Belah ketupat	Kuning	Protected	#	protected
Segitiga	Biru	Package-private	~	(tanpa modifier)

Laporan :

1. Mengapa ada interface dan abstract class, bukan salah satunya saja?

Karena fungsi INterface dan asbtarct itu berbeda

Interface itu ridak membawa kode dan tidak memaksa pewarisan, jadi class apapun bisa masuk ke sistem

Sedangkan abstract itu kerangka kerja bersama. INterface tidak bisa menyimpan state seperti sentCount, dan tanpa abstarct class, kode alur yang sama akan disalin ke Email, SMS dan WhatsApp

2. Apa akibatnya jika send(...) tidak dibuat final?

Akibatnya subclass bisa mengakses send() dan dapat merusak algoritmnta. Jadi dengan adanya final, alurnya terkunci dan subclass hanya mengisi bagian yang memang boleh bervariasi seperti (resolveRecipient, deliver, prepare)

3. Mengapa NotificationService menyimpan List<NotificationChannel>, bukan List<AbstractChannel>?

Jawab : Dengan List<AbstractChannel>, kanal anonim NotificationChannel di TugasApp tidak bisa didaftarkan, karena ia bukan turunan AbstractChannel. Itu bertentangan dengan soal yang meminta kanal log berupa anonymous class NotificationChannel.

T3.

No	Lokasi	Masalah	Dampak nyata	Perbaikan
1	interface StoreOperations, GiftVoucher.ship() dan refund()	Interface (4 method) memaksa voucher mengimplementasikan ship dan refund yang tidak relevan,	Program crash saat voucher.ship(100) dipanggil (exception di output). Kompiler tidak bisa mencegahnya.	Pecah jadi interface kecil: PaymentMethod, RefundablePayment extends PaymentMethod, Shippable. Voucher hanya implements PaymentMethod.
2	Courier.cost(String type, int)	Kurir bukan abstraction yang benar. tarif ditentukan lewat switch pada String, JneCourier kosong, dan abstract tidak punya abstract method.	Salah ketik "EXPRES" (bukan "EXPRESS") jatuh ke default sehingga ongkir menjadi 0.	Buat abstract long ratePerKg() di Courier, tiap kurir (ExpressCourier, dll.) mengisi tarifnya. cost(...) dibuat final
3	Address (setter setStreet, setCity)	Objek mutable dan class tidak final, lalu satu objek Address dipakai bersama dua order.	second.getAddress().setCity(...) ikut mengubah alamat ORD-1 menjadi Bukittinggi.	Jadikan final class Address dengan field private final, tanpa setter. Untuk alamat baru, buat

No	Lokasi	Masalah	Dampak nyata	Perbaikan
				objek baru (withCity(...)).
4	Cart.Item	Dibuat inner class (non-static), padahal tidak memakai data Cart.	Tiap Item menyimpan referensi tersembunyi ke Cart, sehingga boros memori, dan pembuatannya janggal (cart.new Item(...)).	Jadikan static class Item, dibuat dengan new Cart.Item(...).
5	Discount.apply()	Default method memakai instanceof ke class konkret dan angka persen di-hardcode.	Menambah diskon baru berarti mengedit interface. Diskon yang tidak dikenali diam-diam tidak dapat potongan.	Beri abstract method int percent(). Tiap diskon menyebut persentasenya sendiri, apply() tetap default dan tinggal memakai percent().

T4

Tabel 1

Pertanyaan desain	Pilihan	Alasan	Contoh dari proyek
Subclass berbagi field dan konstruktor	**Abstract class**	Interface tidak bisa menyimpan state atau punya konstruktor. Abstract class bisa menyimpan field dan memvalidasinya sekali untuk semua subclass.	ShippingService menyimpan name dan memvalidasinya di konstruktor protected. AbstractChannel menyimpan sentCount dan lastDelivered.
Satu class perlu beberapa kemampuan sekaligus	**Interface**	Java hanya mengizinkan satu extends, tapi class boleh implements banyak interface.	CreditCardPayment implements RefundablePayment, InstallmentPayment.
Rumus bersama yang tidak boleh diubah subclass	**Abstract class**	Method final hanya bisa ada di class, sehingga algoritma terkunci (Template Method). Subclass hanya mengisi bagian yang boleh bervariasi.	ShippingService.calculateCost (final, subclass hanya mengisi ratePerKg). AbstractChannel.send (final).

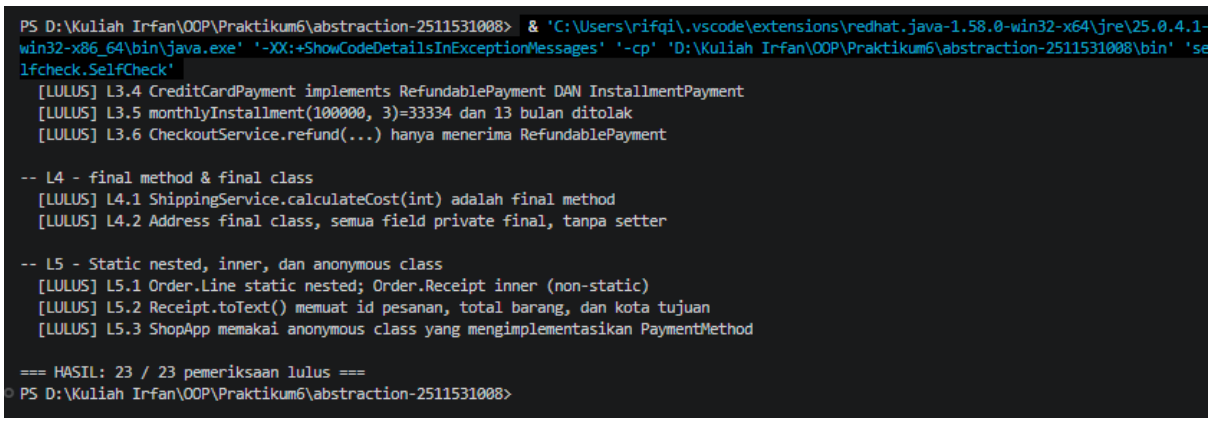
Pertanyaan desain	Pilihan	Alasan	Contoh dari proyek
Kontrak yang perlu ditambah method baru kemudian hari	**Interface** (dengan default method)	Method default memungkinkan kontrak berkembang tanpa merusak class lama (API evolution).	PaymentMethod.adminFee dijadikan default sehingga EWalletPayment tidak perlu diubah.

Tabel 2

Kondisi	Pilihan	Alasan	Contoh dari proyek
Class pembantu tidak membaca data objek luar	**Static nested**	Tidak terikat ke instance luar, jadi lebih ringan dan dibuat tanpa objek luar.	Order.Line (hanya menyimpan product dan quantity).
Objek harus membaca field objek luarnya	**Inner**	Setiap instance terikat ke satu objek luar dan bisa mengakses field-nya langsung.	Order.Receipt membaca orderId, lines, dan shippingAddress milik Order.
Implementasi sekali pakai, tidak perlu nama	**Anonymous**	Tidak perlu membuat file class baru untuk sesuatu yang dipakai di satu tempat.	Voucher PaymentMethod di ShopApp, dan kanal log NotificationChannel di TugasApp.

D. Tangkapan layar TugasCheck, SelfCheck, dan keluaran TugasApp.

TugasCheck



SelfCheck

```
PS D:\Kuliah Irfan\OOP\Praktikum6\abstraction-2511531008> & 'C:\Users\rifqi\.vscode\extensions\redhat.java-1.58.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'D:\Kuliah Irfan\OOP\Praktikum6\abstraction-2511531008\bin' 'selfcheck.TugasCheck'
  [LULUS] T1.3 grandTotal(...) = total + biaya admin (kartu kredit, keranjang contoh, Ekspres)
  [LULUS] T1.4 Diamond policy(): kartu kredit menggabungkan kedua kebijakan

-- T2 - Sistem notifikasi
  [LULUS] T2.1 NotificationChannel interface; AbstractChannel abstract dengan send(...) final
  [LULUS] T2.2 EmailChannel, SmsChannel, WhatsAppChannel adalah subclass AbstractChannel
[SMS -> 081234567890] xxxx
```